DT266

Troubleshoot and Optimize ABAP Cloud Extensions in Cloud ERP

Jürgen Reidl, SAP SE
Andre Fischer, SAP SE
Ingo Braeuninger, SAP SE

May 2026

Today’s scenario – Overview

Today’s scenario – Exercises

Wrap up

Today’s scenario – Overview

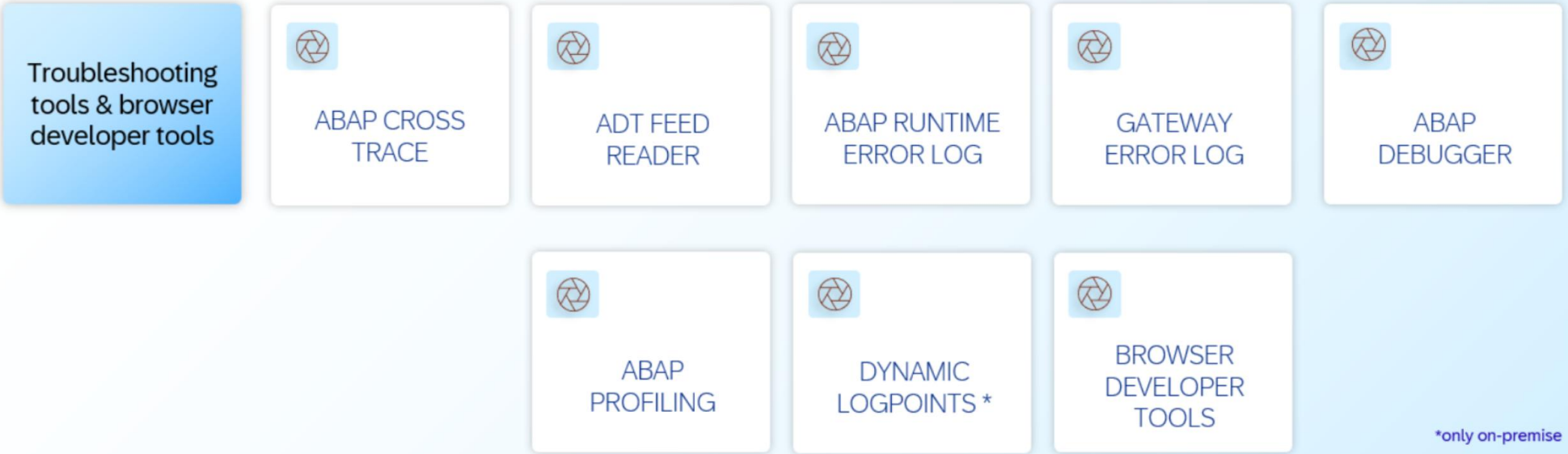
Troubleshooting tools for RAP-based apps

ABAP CROSS TRACE
Designed for RAP-related ABAP functionalities
Provides insight into the RAP runtime framework, can trace data (like a payload trace in SAP Gateway).

FEED READER
Used to read and display feeds from ABAP repository or other native feeds
Includes ABAP short dumps (ABAP Runtime Error Log) , system messages, gateway errors (Gateway Error Log, ...).

ABAP PROFILING (ABAP TRACE)
Designed to trace any executable ABAP code for performance analysis.

Link: https://pages.community.sap.com/topics/abap-testing-analysis/troubleshooting



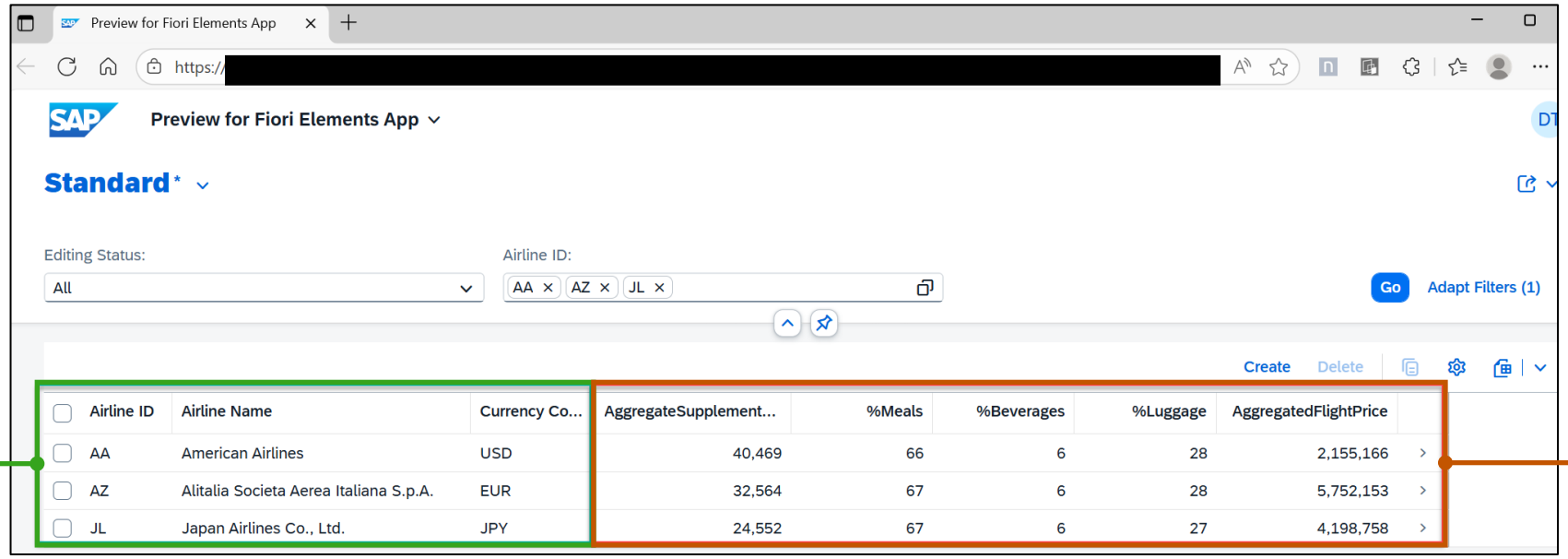
Today's scenario: Enhanced carrier information app

WE HAVE CREATED A CUSTOM EXTENSION OF A RAP APP TO CALCULATE ...

... the total of all corresponding flight prices & the total price of all corresponding supplements (like meal, beverage, luggage)

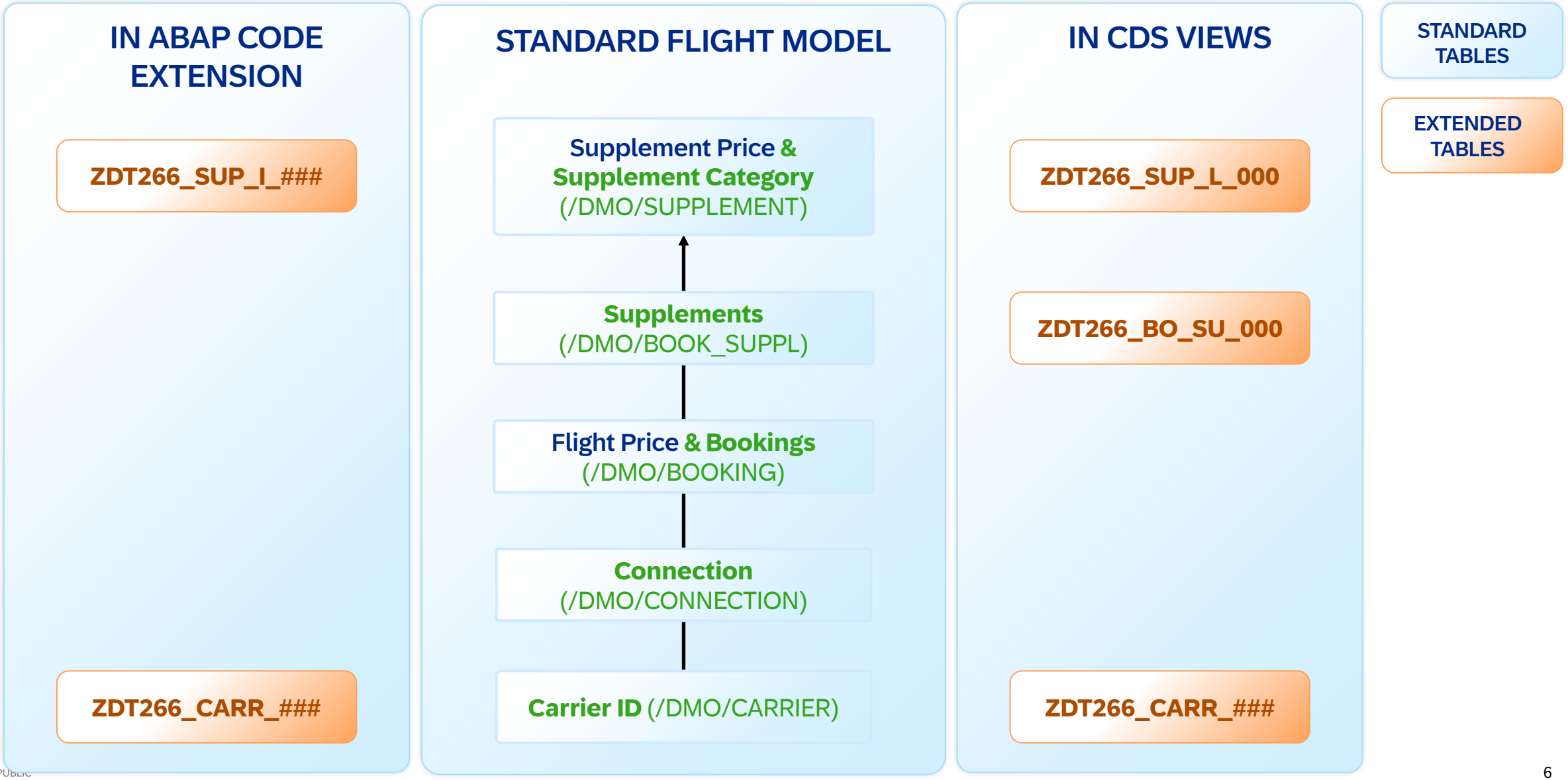
... the contribution in percentage (%Meals, %Beverages, %Luggage) of the supplement categories to the total price of the supplements

GREEN
RAP app directly generated from a database table



RED
Calculated fields in custom extension

Today's scenario: Enhanced carrier information app (tables used)



Today’s scenario – Exercises

DT266 – Troubleshoot and optimize ABAP cloud extensions in cloud ERP

DT266 HANDS-ON EXERCISES

FUNCTIONAL ANALYSIS

- Getting started
- 1 - Usage of debugger in ADT and the feed reader for runtime errors
- 2 - Usage of the memory inspector
- 3 - Usage of the ABAP cross trace

PERFORMANCE ANALYSIS

- 4 - Performance analysis and improvement using ABAP trace and table comparison tool
- 4.1 - Creation and analysis of an ABAP trace
- 4.2 - Use table buffering to improve the performance
- 4.3 - Use secondary index & key to improve the performance

Optional exercises

- 4.4 - Usage of the table comparison tool
- 4.5 - Performance of nested loops

- 5 - SQL trace analysis in SAP HANA SQL analyzer

PREREQUISITES

✅ Set up your ABAP Development Tools for Eclipse

✅ Access to an appropriate ABAP system: (here: APB, see handout)

ACCESS THE EXERCISES

https://github.com/SAP-samples/teched2025-DT266 (Link)

DT266 – Troubleshoot and optimize ABAP cloud extensions in cloud ERP

DT266 HANDS-ON EXERCISES

FUNCTIONAL ANALYSIS

- Getting started
- 1 - Usage of debugger in ADT and the feed reader for runtime errors
- 2 - Usage of the memory inspector
- 3 - Usage of the ABAP cross trace

PERFORMANCE ANALYSIS

- 4 - Performance analysis and improvement using ABAP trace and table comparison tool
  - 4.1 - Creation and analysis of an ABAP trace
  - 4.2 - Use table buffering to improve the performance
  - 4.3 - Use secondary index & key to improve the performance

Optional exercises

- 4.4 - Usage of the table comparison tool
- 4.5 - Performance of nested loops
- 5 - SQL trace analysis in SAP HANA SQL analyzer

ISSUES	TOOLS
Runtime errors	Feed reader & debugger
Out of memory errors	Feed reader, memory inspector & debugger
Only error message	ABAP cross trace
Unexpected modification	ABAP cross trace
ABAP performance	ABAP trace
ABAP performance	ABAP trace
Missing data	Debugger & table comparison tool
CDS view performance	SAP HANA SQL analyzer *

* In Visual Studio Code

The DT266 hands-on exercises – Working on GitHub (1)

Follow the step-by-step instructions provided on GitHub

SAP-samples/teched2025-DT266:

https://github.com/SAP-samples/teched2025-DT266

README | Code of conduct | Contributing | Apache-2.0 license | Security

Exercise Blocks
Getting Started
Getting Started
Exercise Block for Functional Analysis
Exercise 1: Analyze Errors with the Feed Reader
Exercise 2: Usage of the Memory Inspector
Exercise 3: Usage of the ABAP Cross Trace
Exercise Block for Performance Analysis
Exercise 4: Performance Analysis and Improvement using ABAP Trace and Table Comparison Tool • Exercise 4.1 - Creation and Analysis of an ABAP Trace • Exercise 4.2 - Use Table Buffering to Improve the Performance • Exercise 4.3 - Use Secondary Index & Key to Improve the Performance
Optional Exercises for Performance Analysis
Exercise 4: Performance Analysis and Improvement using ABAP Trace and Table Comparison Tool • Exercise 4.4 - Usage of the Table Comparison Tool • Exercise 4.5 - Performance of Nested LOOPs
Exercise 5: SQL Trace Analysis in SAP HANA SQL Analyzer

Home - DT266

Exercise 1: Analyze Errors with the Feed Reader

Introduction

In the previous Getting Started Section you've made yourself familiar with the data model and the ABAP code (see Getting Started).

In this exercise we focus on error analysis of runtime errors due to an exception thrown during the processing. You learn to use the *FEED_READER* for error analysis. We will do a further deep dive in the error analyis using the *ABAP_Debugger*. Finally we solve the issue by a small change of ABAP code.

Exercises

- 1.1 - Runtime Error Analysis with the Feed Reader

Summary:

- Summary & Next Exercise

Exercises

Exercise 1.1: Runtime Error Analysis with the Feed Reader

^Top of page

> Here we have 3 steps:
>
> 1. Perform the step for two different Airline IDs, where we get a runtime error for one of them.
> 2. Call the Feed Reader and analyze the error.
> 3. Correct the error.

> For this use `ZDT266_###` ZDT266, where you replace `###` with your suffix.
> Now we perform the step for two different Airline IDs, **AA** and **AC**.

▶ Click to expand!

Summary & Next Exercise

^Top of page

Now that you've...

- used the *FEED READER* for error analyis and correction of coding,
- used the Debugger and Breakpoints to analyze the content of the session variables

Congratulations! 🎉

In this hands-on exercise group, you have hopefully have some more insights into error analysis capabilities such as analyzing Runtime Error dumps!

The DT266 hands-on exercises – Working on GitHub (2)

main | teched2025-DT266 / exercises / ex04 / README.md | Top

Preview | Code | Blame | Raw

Introduction

Important

Prerequisite for this exercise is that you have implemented **both specific subsequent** code changes

- Code change of exercise 2.1 - Coding Change for Reading the Supplements
- Code change of exercise 2.3 - Correction of the ABAP Code

In case they are not yet implemented a code snippet is provided below to speed up the process: In this case delete the complete current source code in the class `ZCL_DT266_CARR_EXTENSION_###`, insert the code snippet provided below ( ), and replace all occurrences of the placeholder `###` with your personal suffix using the ADT function ***Replace All*** (***Ctrl+F***).

▼ Click to expand and replace the source code if not performed the previous exercises!

- Make use of the *Copy Raw Content* ( ) function to copy the provided code snippet.
- Replace all occurrences of the placeholder `###` with your personal suffix using the ADT function ***Replace All*** (***Ctrl+F***).

```
CLASS zcl_dt266_carr_extension_### DEFINITION
PUBLIC
FINAL
CREATE PUBLIC .
```

Exercises

Exercise 4.1: Creation and Analysis of an ABAP Trace

^Top of page

Here we learn to create and profile an ABAP trace. And we learn to analyze the ABAP trace to identify improvement potential.

- The performance issues discovered will be further analyzed in exercises 4.2, 4.3 and 4.4.
- There we will discuss and implement solutions for them and will significantly improve the runtime.

▶ Click to expand!

Outline

Filter headings

Exercise 4: Analyze Performance Issues with ABAP Trace and Table Comparison Tool
Introduction
Exercises
Optional Exercise:
Summary:
Exercises
Exercise 4.1: Creation and Analysis of an ABAP Trace
Exercise 4.2: Use Table Buffering to Improve the Performance
Exercise 4.3: Use Secondary Index & Key to Improve the Performance
Exercise 4.4: Usage of the Table Comparison Tool
Exercise 4.5: Performance of Nested LOOPs
Summary & Next Exercise
License

Use outline for a **quick overview**

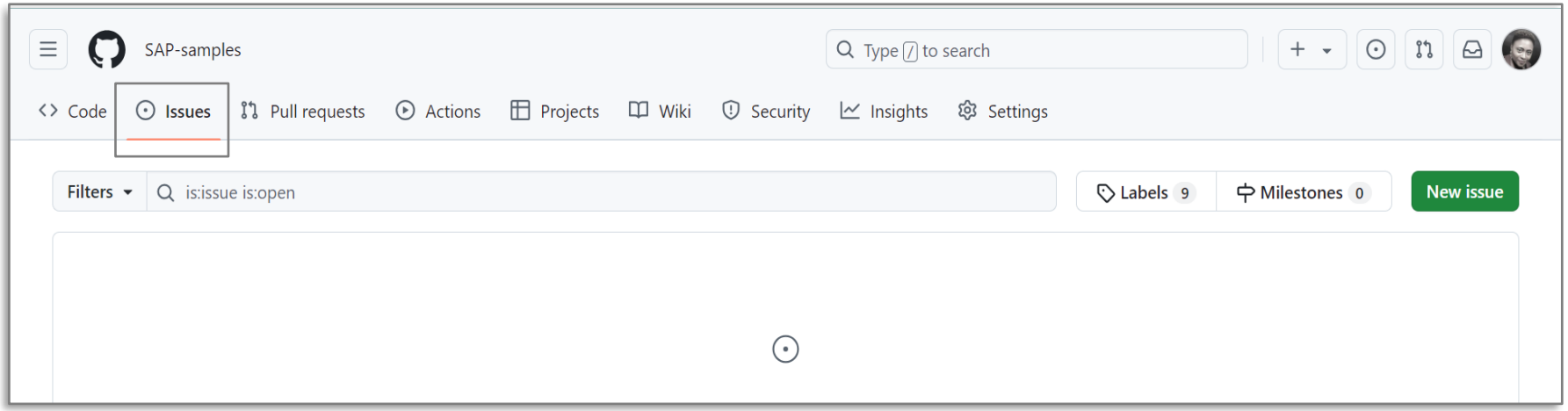
Replace all occurrences of ### with your suffix

Copy raw content

Expand to see details

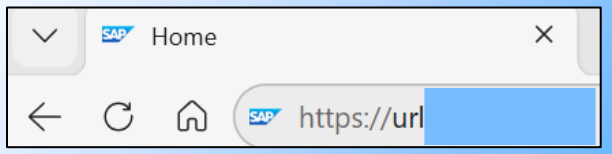
The DT266 hands-on exercises – Working on GitHub (3)

Open GitHub Issues to get help from the SAP experts

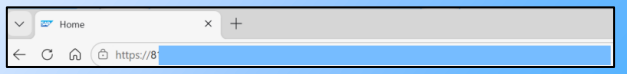


How to get the Logon URL

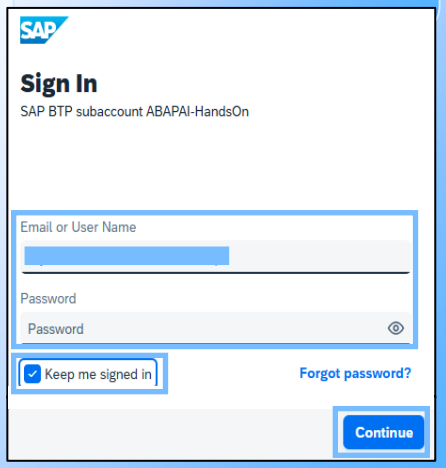
(1) Enter short URL from the Hand-Out in the address bar of your browser:



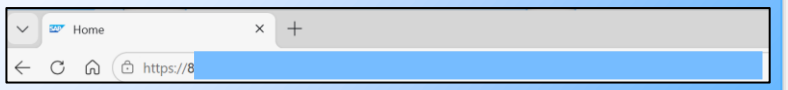
to get the complete URL



(2) Enter user name and password

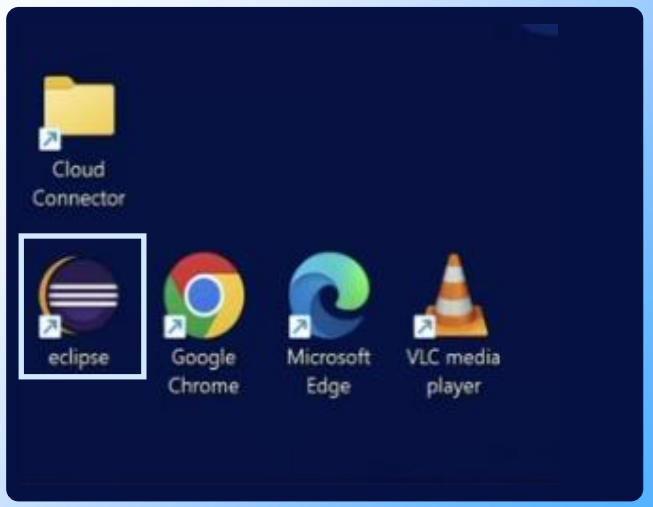


(3) Get and copy the complete URL you receive

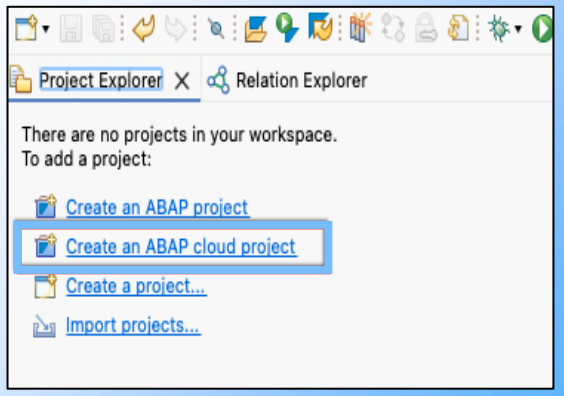


How to logon in Eclipse

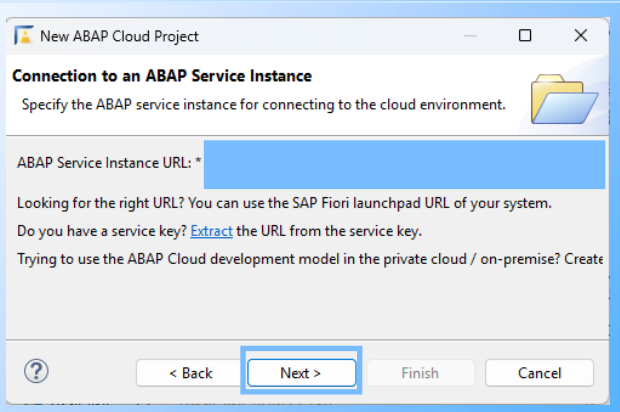
(4) Double-click the eclipse icon on the desktop to start it



(5) Create a new ABAP Cloud Project

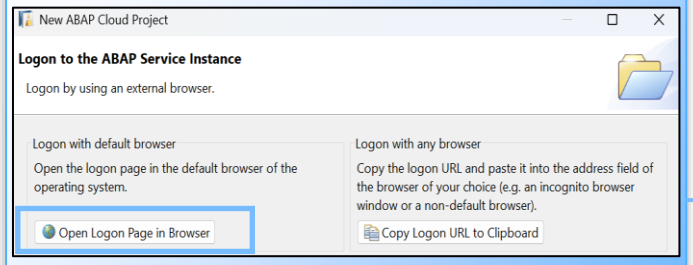


(6) Enter the complete URL you receive from the short URL in the Hand-Out as described in steps **(1) – (3)**

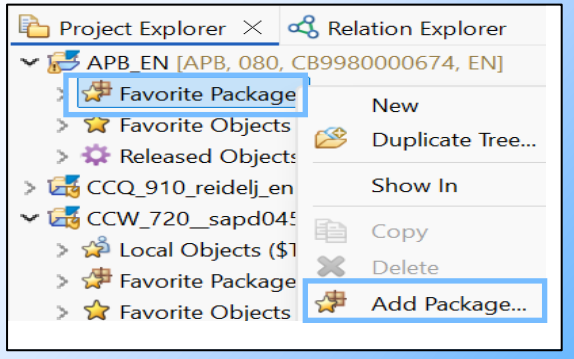


How to access your package in Eclipse

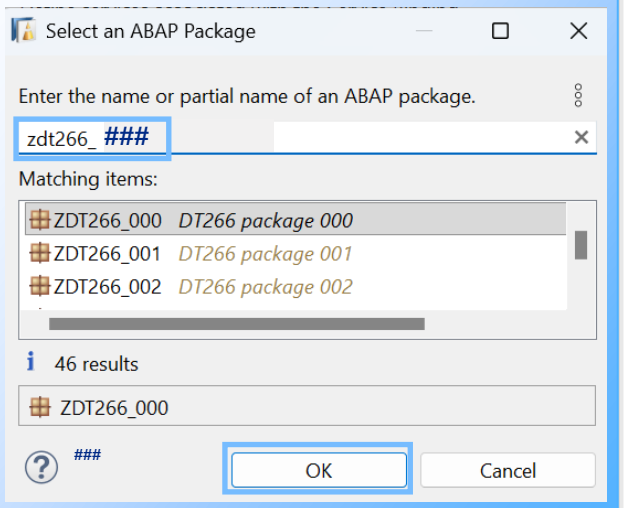
(8) Click on open logon page in browser



(9) Add your package to favorite packages



(10) Enter with your group number



LET'S GET STARTED!

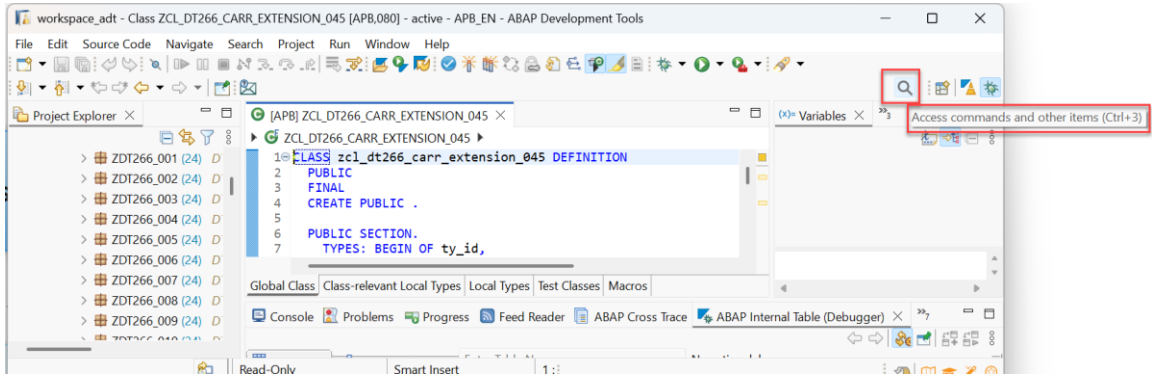
Move at your own pace!

Feel free to ask questions!

ACCESS THE EXERCISES

https://github.com/SAP-samples/teched2025-DT266 (Link)

ACCESS THE TOOLs



Wrap up

THANK YOU!

Jürgen Reidl
Development Expert

Dietmar-Hopp Allee 16
69190 Walldorf
SAP SE

SAP Bring out your best.

© 2026 SAP SE or an SAP affiliate company. All rights reserved. See Legal Notice on www.sap.com/legal-notice for use terms, disclaimers, disclosures, or restrictions related to this material.